

Web GIS Laboratory

District Explorer: building a web map from a shapefile

Dr. Shahid Nawaz Khan

Institute of Geographical Information Systems

School of Civil and Environmental Engineering

National University of Sciences and Technology, Islamabad

MS Remote Sensing and GIS

Duration	3 hours
Covers	Both preceding lectures
Software	QGIS, Visual Studio Code, a browser, Git
Deliverable	A published map and a measurement report

Contents

1	Before you begin	2
1.1	What you will build	2
1.2	Learning outcomes	3
1.3	Structure and timing	3
1.4	What you need	4
2	Set up the project	4
2.1	Create the folder	4
2.2	Create the repository	5
2.3	Create the three files	5
3	Prepare the data in QGIS	5
3.1	Inspect what you were given	5
3.2	Version A: a straight conversion	6
3.3	Version B: reduced coordinate precision	6
3.4	Versions C and D: simplified geometry	7
3.5	Record your measurements	7
4	Read the data as text	8
4.1	Open it	8
4.2	Find the structure	8
4.3	Check the coordinate order	8
4.4	Check that the rings close	9

5	Build the page	9
5.1	index.html	9
5.2	style.css	10
5.3	View it	11
6	Load the data with JavaScript	11
6.1	Create the map	11
6.2	Keep the view on Pakistan	12
6.3	Fetch the file	12
6.4	Try the array methods	13
7	Draw it with Leaflet	14
7.1	Add the layer	14
7.2	Check the network activity	15
8	Classify and add a popup	15
8.1	Shade by population	15
8.2	Add a popup	16
8.3	Add a legend	17
9	Filter with a control	17
9.1	Fill the drop-down from the data	18
9.2	React to the user	18
10	Measure the coordinate systems	19
10.1	Produce two projected copies	19
10.2	Compute areas in each	19
10.3	Compare	20
10.4	Answer these	20
11	Publish and submit	20
11.1	Commit and push	20
11.2	Turn on hosting	21
11.3	Write the report	21
11.4	Questions	21
11.5	Marking	22
	Appendix A: Troubleshooting	23
	Appendix B: Reference	23

1 Before you begin

1.1 What you will build

A working web map of Pakistan's districts, published at a public URL. The finished page lets a user pick a province from a drop-down, redraws the map with only those districts, shades each district by population, and shows its attributes when clicked.

You will also produce a short measurement report comparing file sizes, load times and computed areas, using your own numbers rather than the figures quoted in the lecture.

1.2 Learning outcomes

By the end of this laboratory you will be able to:

1. Convert a shapefile to GeoJSON and control its size.
2. Explain the effect of coordinate precision and simplification on file size, and choose values appropriate to a map's scale.
3. Read a GeoJSON file as text and identify its structure and its errors.
4. Request a file from JavaScript and inspect it in the browser console.
5. Draw, style, classify and filter a vector layer with Leaflet.
6. Connect an HTML control to a map through an event listener.
7. Measure the difference between an area computed in Web Mercator and one computed in a projected system.
8. Publish a static site and read its network activity.

1.3 Structure and timing

Parts 1 to 7 are the core of the laboratory and everyone is expected to finish them. Parts 8 and 9 are extensions. If you run short of time, complete Part 10 first so that your work is published, then return to the extensions.

Part	What you do	Time
1	Set up the project	15 min
2	Prepare the data in QGIS	30 min
3	Read the data as text	15 min
4	Build the page	20 min
5	Load the data with JavaScript	20 min
6	Draw it with Leaflet	25 min
7	Classify and add a popup	20 min
8	Filter with a control (<i>extension</i>)	25 min
9	Measure the coordinate systems (<i>extension</i>)	20 min
10	Publish and submit	20 min

1.4 What you need

Item	Requirement
QGIS	Version 3.28 or later
Visual Studio Code	With the Live Server extension installed
Browser	Chrome or Edge, for the developer tools
Git	Installed and configured with your name and email
Account	A GitHub account
Data	The districts shapefile from LMS, about 24 MB zipped

Where the data comes from, and what is in it

The layer is the Pakistan Common Operational Dataset for administrative boundaries, published by OCHA on the Humanitarian Data Exchange and sourced from the World Food Programme. The download bundle covers four levels: country, province, district and tehsil. Administrative level 2 is the district layer, with 160 features. That is the only one you need.

This handout uses the field names `district`, `province`, `pop_2023` and `area_km2`. The original dataset names them differently, using `adm2_name`, `adm1_name`, `adm2_pcode` and `area_sqkm`, and it has no population at all. The copy on LMS has been renamed, and `pop_2023` has been joined to it from the 2023 census by matching district names. Open the attribute table in QGIS before you start and confirm. If the names differ, substitute yours everywhere they appear in the code.

The 2023 census did not cover Azad Kashmir or Gilgit-Baltistan, so the 24 districts there have no population value. Lehri, in Balochistan, is not listed separately in the census tables either. Expect 25 empty values, and see Part 7 for how the map handles them.

2 Set up the project

2.1 Create the folder

Open PowerShell and run:

```
cd C:\
mkdir webgis-lab
cd C:\webgis-lab
mkdir data
code .
```

Where the folder goes

Use `C:\webgis-lab`. Do not use Documents, Desktop, or any folder inside OneDrive. Those folders are synced, and the sync process locks files while a web server is reading them. Spaces in the path cause a separate set of problems later.

2.2 Create the repository

1. On github.com, create a new repository named `webgis-lab-<your-roll-number>`.
2. Set it to **Public**. This is required for free hosting.
3. Do not tick any of the initialise options.
4. Connect your folder to it:

```
git init
git branch -M main
git remote add origin https://github.com/<your-username>/webgis-lab-<roll>.git
```

2.3 Create the three files

In the VS Code explorer, create these in `C:\webgis-lab`:

File	Holds
<code>index.html</code>	The structure of the page
<code>style.css</code>	How the page looks
<code>app.js</code>	What the page does

Keeping the three languages in three files is the normal arrangement once a page grows beyond a few lines. The HTML file will link to the other two.

Checkpoint 1

A screenshot of VS Code showing the folder `C:\webgis-lab` with `index.html`, `style.css`, `app.js` and an empty data folder.

3 Prepare the data in QGIS

This part produces four versions of the same layer. Record the size of each one as you go, because the comparison is part of your report.

3.1 Inspect what you were given

1. Unzip the shapefile into a working folder **outside** `C:\webgis-lab`. Raw data is not committed to the repository.
2. Load it into QGIS and open the attribute table. Note the field names and their types.

3. Open **Layer Properties**, then **Information**, and record the coordinate reference system and the feature count.
4. In your file manager, note the total size of all the shapefile components together.

Why the field types matter A population column stored as text rather than as a number will sort and compare alphabetically, so 9 will appear greater than 1500. This breaks any classification built on it and produces no error. Check the type now rather than after your choropleth looks wrong.

3.2 Version A: a straight conversion

Right-click the layer, choose **Export**, then **Save Features As**.

Setting	Value
Format	GeoJSON
File name	districts_A.geojson
CRS	EPSG:4326, WGS 84
Coordinate precision	Leave at the default

Record the resulting file size.

Always export in EPSG:4326

GeoJSON assumes longitude and latitude in WGS 84. Exporting in any other system produces a file that is syntactically valid and silently placed in the wrong part of the world. If your data arrived in a projected system, this export step is also the reprojection step.

3.3 Version B: reduced coordinate precision

Export again. This time, in the **Layer Options** section of the dialog, set `COORDINATE_PRECISION` to 5. Name it `districts_B.geojson` and record the size.

Decimal places	Resolves to
3	110 m
4	11 m
5	1.1 m
6	11 cm

Five decimal places locate a point to about a metre. A district boundary digitised from 30 m imagery does not justify more than that, and writing more both wastes bandwidth and overstates the accuracy of the data.

3.4 Versions C and D: simplified geometry

Use **Vector**, then **Geometry Tools**, then **Simplify**. Run it twice on the original layer.

Version	Tolerance	Intended use
C	0.001 degrees, about 100 m	A provincial or national view
D	0.01 degrees, about 1 km	A thumbnail or an overview map

Export each result to GeoJSON with precision 5, as `districts_C.geojson` and `districts_D.geojson`. Record both sizes.

Gaps along shared borders

Simplifying adjacent polygons independently moves their shared borders apart, leaving slivers and gaps. In the Simplify dialog, keep **Preserve topology** enabled. If you see hairline gaps between districts later, this is the cause.

3.5 Record your measurements

Fill this in. These are your numbers, not the lecture's.

Version	What was done	Size	% of A
Shapefile	The adm2 .shp, .shx, .dbf, .prj and .cpg added together		n/a
A	Straight conversion		
B	Precision 5		
C	Simplified 100 m, precision 5		
D	Simplified 1 km, precision 5		

Measure the layer, not the download The bundle you downloaded holds four administrative levels plus a lines layer, so its total size tells you nothing useful. Add up only the adm2 components. Those are the files your GeoJSON was made from, and they are the only fair comparison.

For reference, the published download sizes are 24.3 MB for the shapefile bundle and 27.7 MB for the same data as GeoJSON. Note which is larger, and say why in your report.

Copy version C into your project as the one the map will use:

```
copy districts_C.geojson C:\webgis-lab\data\districts.geojson
```

Checkpoint 2

The completed measurement table, and a screenshot showing versions A and D drawn together in QGIS so the loss of detail is visible.

4 Read the data as text

GeoJSON is a text format, which means you can inspect it directly. This is the main practical advantage it has over a shapefile.

4.1 Open it

Open `data/districts.geojson` in VS Code. Press `Shift+Alt+F` to format it, which puts each item on its own line and makes the structure visible.

Format a copy, not the file you will serve

Formatting inserts spaces and line breaks, which can add 30 percent to the file size. Format it to read it, then undo with `Ctrl+Z` before saving.

4.2 Find the structure

Answer these from the file itself:

1. What is the type of the outermost object?
2. How many entries are in the `features` array? VS Code shows the count if you select the array.
3. Take the first feature. What is its geometry type?
4. How many levels of square bracket are there before the first number in its `coordinates`?
5. List the keys inside its `properties`.

4.3 Check the coordinate order

Find the first coordinate pair in the file.

What the numbers should look like Latitude in Pakistan runs from about 24 to 37. Longitude runs from about 61 to 77.

GeoJSON writes longitude first, so the first number of each pair should be in the sixties or seventies, and the second in the twenties or thirties. If they are the other way round, the file is wrong and every feature will be drawn in the wrong place.

4.4 Check that the rings close

Take any polygon. Compare the first coordinate pair in its outer ring with the last one. They must be identical. A ring that does not close is invalid, and some renderers will reject the whole file rather than that one feature.

Checkpoint 3

Your five answers from 4.2, plus the first coordinate pair you found and a sentence confirming the order is correct.

5 Build the page

5.1 index.html

index.html, complete

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>District Explorer</title>

  <link rel="stylesheet"
        href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>District Explorer</h1>
    <p class="subtitle">Districts of Pakistan, shaded by population</p>
  </header>

  <div id="panel">
    <label for="province">Province</label>
    <select id="province">
      <option value="all">All provinces</option>
    </select>
    <span id="count">loading...</span>
  </div>

  <div id="map"></div>

  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

Why the script tags sit at the end of the body A script runs as soon as the browser reaches it. If `app.js` ran before the `<div id="map">` existed, it could not find it, and Leaflet would report that the map container was not found. Placing the scripts last guarantees the elements exist first.

The Leaflet tag must come before `app.js`, because your code uses `L`, which Leaflet defines.

5.2 style.css

style.css, complete

```
body {
  font-family: system-ui, sans-serif;
  margin: 0;
  color: #23373b;
  background: #f4f6f6;
}

header {
  padding: 18px 24px 10px;
}

h1 {
  margin: 0;
  font-size: 24px;
}

.subtitle {
  margin: 4px 0 0;
  color: #5b6a6e;
}

#panel {
  padding: 10px 24px 14px;
  display: flex;
  align-items: center;
  gap: 12px;
}

#count {
  color: #5b6a6e;
  font-size: 14px;
}

#map {
  height: calc(100vh - 150px);
  width: 100%;
}

.legend {
  background: white;
  padding: 8px 10px;
  line-height: 20px;
  border-radius: 4px;
  box-shadow: 0 1px 4px rgba(0,0,0,.25);
  font-size: 13px;
}
```

```
}

.leaflet i {
  width: 16px;
  height: 16px;
  float: left;
  margin-right: 8px;
  opacity: .85;
}
```

The height rule

`#map` must have a height. An empty `<div>` has zero height, and Leaflet will draw a map zero pixels tall into it with no error message and a clean console.

`calc(100vh - 150px)` means the full window height minus the space taken by the header and panel. If your map does not appear, this line is the first thing to check.

5.3 View it

Right-click in the editor and choose **Open with Live Server**. Confirm that the address bar reads `http://127.0.0.1:5500` and not `file:///C:/...`

Why this matters now rather than later In Part 6 the page will request a file with `fetch`. A browser blocks that request for pages opened directly from the file system. Opened that way, your data will silently fail to load and the error will not explain why.

Checkpoint 4

A screenshot showing the header and the empty grey map area, with the address bar visible on `127.0.0.1`.

6 Load the data with JavaScript

6.1 Create the map

app.js, first version

```
const map = L.map('map').setView([30.3753, 69.3451], 5);

L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
  maxZoom: 19,
  attribution: '&copy; OpenStreetMap contributors'
}).addTo(map);
```

Save and reload. You should have a pannable map centred on Pakistan.

Coordinate order, again `setView` takes `[latitude, longitude]`. Your GeoJSON file writes `[longitude, latitude]`. The two conventions are both established and neither is going to change, so the only defence is knowing which one you are addressing.

Leaflet reads GeoJSON correctly on its own. It is only the coordinates you type by hand that need care.

6.2 Keep the view on Pakistan

As it stands, a user can drag the map to Europe or zoom out to the whole world. Replace the first line of `app.js` with this:

app.js, replacing the `L.map` line

```
const pakistanLimits = L.latLngBounds([21.5, 58.5], [39.0, 80.5]);

const map = L.map('map', {
  maxBounds: pakistanLimits,
  maxBoundsViscosity: 1.0,
  minZoom: 5
}).setView([30.3753, 69.3451], 5);
```

Option	Effect
<code>maxBounds</code>	The box the view may not leave
<code>maxBoundsViscosity</code>	1.0 stops dead at the edge; lower values let the map stretch past it and spring back
<code>minZoom</code>	Stops the user zooming out beyond the country

Pakistan spans about 23.6 to 37.1°N and 60.9 to 77.9°E. The limits add about two degrees on each side, so that border districts are not pressed against the edge of the window. Note the order: `L.latLngBounds` takes the south-west corner first, then the north-east, each as `[latitude, longitude]`.

6.3 Fetch the file

Add this below the tile layer.

app.js, continued

```
let districts = null;

async function loadData() {
  const response = await fetch('data/districts.geojson');

  if (!response.ok) {
    console.error('Request failed with status', response.status);
    return;
  }
}
```

```

    districts = await response.json();

    console.log('features:', districts.features.length);
    console.log('first feature:', districts.features[0]);
    console.log('properties:', districts.features[0].properties);
  }

  loadData();

```

Open the Console tab in the developer tools and reload.

1. Confirm the feature count matches what QGIS reported.
2. Expand `first feature` and walk down to its coordinates.
3. Read the property names from `properties`, and confirm they match the ones you will use in the next part.

What `await` is doing The file has to travel over the network, which takes time and can fail. `await` pauses until the response arrives. Without it, the next line would run before the data existed, and `districts` would still be `null`.

The `response.ok` check catches the case where the server answered but the file was not found. A 404 means the path is wrong; look at your own code. A 500 means the server failed; look at the server.

6.4 Try the array methods

With the page loaded, type these straight into the Console. They operate on real data and the results print immediately.

```

districts.features.length

districts.features.map(f => f.properties.district).slice(0, 10)

districts.features.filter(f => f.properties.province === 'Punjab').length

[...new Set(districts.features.map(f => f.properties.province))].sort()

```

Method	Does
<code>map</code>	Turns every feature into something else, here its name
<code>filter</code>	Keeps the features that pass a test
<code>Set</code>	Removes duplicates, giving the list of distinct provinces

The last line is the one you will use in Part 8 to fill the drop-down, so note what it returns.

Checkpoint 5

A screenshot of the Console showing the feature count, the expanded first feature, and the list of distinct provinces.

7 Draw it with Leaflet

7.1 Add the layer

Add a draw function and call it at the end of `loadData`.

app.js, continued

```
let layer = null;

function draw(features) {
  if (layer) {
    map.removeLayer(layer);
  }

  layer = L.geoJSON(
    { type: 'FeatureCollection', features: features },
    {
      style: {
        color: '#ffffff',
        weight: 1,
        fillColor: '#8c9ea3',
        fillOpacity: 0.75
      }
    }
  ).addTo(map);

  if (features.length > 0) {
    map.fitBounds(layer.getBounds());
  }

  document.getElementById('count').textContent =
    features.length + ' districts';
}
```

Then at the end of `loadData`, after the `console.log` lines:

```
draw(districts.features);
```

In Part 8 you will add one more line immediately above that one, so the finished end of `loadData` reads:

```
fillProvinceList();
draw(districts.features);
```

Why the two guards are there `removeLayer` runs first because `L.geoJSON(...).addTo(map)` adds a layer, it does not replace one. Without it, filtering would stack layers on top of each other and the map would slow down with every change.

`fitBounds` is skipped when the list is empty, because an empty layer has no bounds and Leaflet throws `Bounds are not valid`, which stops the rest of the function from running.

7.2 Check the network activity

Open the **Network** tab, tick **Disable cache**, and reload.

1. Find the request for `districts.geojson`. Record its size and the time it took.
2. Count the requests ending in `.png`. These are map tiles.
3. Click one tile and open its **Preview**. You are looking at a single 256 by 256 pixel square of map.
4. Read the three numbers in that tile's URL and identify which is the zoom level.
5. Pan the map and watch new tile requests arrive.

Throttling the connection At the top of the Network tab there is a throttling drop-down, usually set to **No throttling**. Set it to **Slow 3G** and reload.

The load time you now see is closer to what a user on a mobile connection experiences. Record it. Then try the same with `districts_A.geojson` substituted in, and record that too. Set the throttle back to **No throttling** afterwards.

Checkpoint 6

A screenshot of the Network tab showing the GeoJSON request with its size, and a selected tile request with its Preview visible.

8 Classify and add a popup

8.1 Shade by population

Replace the fixed `fillColor` with a function. Add this above `draw`.

app.js, continued

```
function show(value) {
  if (value === null || value === undefined || value === '') {
    return 'not recorded';
  }
  return typeof value === 'number' ? value.toLocaleString() : value;
}

function colourFor(population) {
  if (population === null || population === undefined) return '#c9cfd1';
```

```

    if (population > 3000000) return '#7f2704';
    if (population > 1500000) return '#d94801';
    if (population > 750000) return '#f16913';
    if (population > 300000) return '#fd8d3c';
    return '#fdd0a2';
  }

  function styleFor(feature) {
    return {
      color: 'ffffff',
      weight: 1,
      fillColor: colourFor(feature.properties.pop_2023),
      fillOpacity: 0.8
    };
  }
}

```

Then change the style option inside draw to use it:

```
style: styleFor,
```

This is a classification The chain of `if` statements is a manual classification with five classes and four break values. Changing the breaks changes the map's message, which is why the breaks belong in your report alongside the map.

If every district comes out the same colour, inspect the values themselves. A plain numeric string such as "1500000" is converted to a number by the comparison and classifies correctly. A value with a thousands separator, a stray space, or a placeholder such as "NA" cannot be converted, so every comparison is false and the feature falls through to the last line.

Missing is not the same as small The first line of `colourFor` catches districts with no population value and paints them grey. Without it, a missing value fails every comparison and lands in the lowest class, so Azad Kashmir and Gilgit-Baltistan would appear as thinly populated rather than as unrecorded. A choropleth should never let the reader mistake "no data" for "low".

8.2 Add a popup

Add an `onEachFeature` option to the `L.geoJSON` call, directly after `style`.

inside the options object in draw

```

onEachFeature: function (feature, featureLayer) {
  const p = feature.properties;
  featureLayer.bindPopup(
    '<strong>' + show(p.district) + '</strong><br>' +
    'Province: ' + show(p.province) + '<br>' +
    'Population: ' + show(p.pop_2023) + '<br>' +
    'Area: ' + show(p.area_km2) + ' km<sup>2</sup>'
  );
}

```

This is a callback You are not running that function. You are handing it to Leaflet, which runs it once for every feature as the layer is built. The same idea appears again in Part 8 with the drop-down.

Why the popup goes through `show`

Calling `.toLocaleString()` straight on an attribute fails if that attribute is `null`, or if the field is named differently in your copy of the data. The failure is not limited to one feature: it happens while Leaflet is building the layer, so the whole map fails to draw and the console reports `Cannot read properties of null`.

`show` returns a readable placeholder instead, so one missing value costs you one popup line rather than the entire map.

8.3 Add a legend

`app.js`, at the end

```
const legend = L.control({ position: 'bottomright' });

legend.onAdd = function () {
  const div = L.DomUtil.create('div', 'legend');
  const breaks = [0, 300000, 750000, 1500000, 3000000];
  const labels = ['under 300k', '300k to 750k', '750k to 1.5m',
    '1.5m to 3m', 'over 3m'];

  for (let i = 0; i < breaks.length; i++) {
    div.innerHTML +=
      '<i style="background:' + colourFor(breaks[i] + 1) + '"></i> ' +
      labels[i] + '<br>';
  }
  div.innerHTML +=
    '<i style="background:' + colourFor(null) + '"></i> no census data';
  return div;
};

legend.addTo(map);
```

Checkpoint 7

A screenshot of the shaded map with the legend visible and one popup open.

9 Filter with a control

Extension exercise

9.1 Fill the drop-down from the data

Add this function, and call it from `loadData` before `draw`.

app.js, continued

```
function fillProvinceList() {
  const select = document.getElementById('province');

  const provinces = [...new Set(
    districts.features.map(f => f.properties.province)
  )].sort();

  for (const name of provinces) {
    const option = document.createElement('option');
    option.value = name;
    option.textContent = name;
    select.appendChild(option);
  }
}
```

Building the list from the data The drop-down is not typed by hand. It is derived from the file, so it stays correct if the data changes. This is the same `map` and `Set` combination you tried in the Console in Part 6.

9.2 React to the user

app.js, continued

```
document.getElementById('province')
  .addEventListener('change', function (event) {
    const chosen = event.target.value;

    const subset = chosen === 'all'
      ? districts.features
      : districts.features.filter(
        f => f.properties.province === chosen
      );

    draw(subset);
  });
```

Reload and change the drop-down. The map should redraw with only the chosen province, zoom to it, and update the district count.

Piece	Role
<code>getElementById</code>	Finds the element in the page
<code>addEventListener</code>	Registers a function to run later
<code>'change'</code>	The event to wait for
<code>event.target.value</code>	What the user chose
<code>filter</code>	Keeps only the matching features

Every interactive control in a dashboard is a variation on these five lines: find an element, listen for an event, filter the data, redraw.

Checkpoint 8

Two screenshots, one showing all provinces and one showing a single province selected, with the district count visible in both.

10 Measure the coordinate systems

Extension exercise

The lecture claimed that an area computed in Web Mercator at this latitude is about 40 percent too large. This part tests that claim with your own data.

10.1 Produce two projected copies

In QGIS, export the original layer twice.

Export as	CRS	Is
d_3857	EPSG:3857	Web Mercator, what the map is drawn in
d_32643	EPSG:32643	UTM zone 43N, a projected system for this region

10.2 Compute areas in each

On each layer, open the **Field Calculator** and create a new decimal field called `area_calc` with the expression:

```
$area / 1000000
```

`$area` is computed in the layer's own coordinate system, so the two layers will give different answers for the same district.

10.3 Compare

Pick five districts spread from the south of the country to the north. Record their values.

District	Latitude	3857 km ²	UTM km ²	% difference

10.4 Answer these

1. Does the percentage difference grow or shrink as you move north?
2. The predicted inflation factor is $1/\cos^2(\text{latitude})$. Compute it for one of your districts and compare it with the measured difference.
3. Which of the two figures would you put in a report, and why?

The working rule

Store data in EPSG:4326. Display it in EPSG:3857, which the mapping library handles for you. Measure in a projected system such as UTM zone 42N or 43N. Never report an area or a distance computed in Web Mercator.

Checkpoint 9

The completed comparison table and your three answers.

11 Publish and submit

11.1 Commit and push

```
cd C:\webgis-lab
git add -A
git commit -m "District Explorer: map, classification, filter"
git push -u origin main
```

A browser window will open Git for Windows authenticates through your browser rather than asking for a password in the terminal. Approve it once and it remembers. If anything asks you to type your GitHub password into PowerShell, stop and ask for help.

11.2 Turn on hosting

1. On the repository page, open **Settings**, then **Pages**.
2. Under Source, choose **Deploy from a branch**.
3. Set the branch to `main` and the folder to `/ (root)`. Save.
4. Wait one to three minutes, then refresh until the live URL appears.

Open the URL on your phone, using mobile data rather than campus wifi. The map is now being served over the public internet.

11.3 Write the report

Create `report.md` in the repository, containing:

1. Your nine checkpoint screenshots.
2. The file size table from Part 3.
3. The load times you measured with and without throttling.
4. The classification breaks you used, and one sentence on why.
5. The coordinate system comparison table from Part 9, if you completed it.
6. Your answers to the questions below.

11.4 Questions

1. Version D is far smaller than version A. Give one situation in which version D would be the wrong choice.
2. The page makes one request for your GeoJSON file and dozens for tiles. Explain the difference in how those two kinds of data reach the browser.
3. Why does the map container need a height when the heading does not?
4. `draw` removes the existing layer before adding a new one. What would go wrong if it did not?
5. A classmate's districts all appear in the same colour. List two possible causes and how you would tell them apart.
6. You open the page by double-clicking `index.html` instead of using Live Server. The basemap appears but the districts do not. Explain why.

11.5 Marking

Item	Marks
Published map that loads, with shading, legend and popups	30
File size measurement table, with correct method	15
Network measurements, including the throttled figures	10
Nine checkpoint screenshots	15
Answers to the six questions	20
Repository with a readable commit history	10
Extensions in Parts 8 and 9	+10 bonus

Submit two links on LMS: your repository and your live page.

Appendix A: Troubleshooting

Symptom	Cause and fix
Nothing where the map should be	The container has no height. Check the <code>#map</code> rule in <code>style.css</code>
Grey map area, no tiles	No internet, or the tile host is blocked. Check that you can open <code>unpkg.com</code> in a tab
<code>L</code> is not defined	The Leaflet script tag is missing, misspelled, or placed after <code>app.js</code> . Leaflet must load first
Map container not found	The id in the HTML and the id in <code>L.map()</code> do not match. Both must be exactly <code>map</code>
Basemap loads, districts do not	Opened from the file system. Use Live Server. Check the address bar reads <code>127.0.0.1</code>
404 on <code>districts.geojson</code>	Wrong path. The file must be at <code>data/districts.geojson</code> relative to <code>index.html</code>
Cannot read properties of null	<code>draw</code> ran before the data arrived. Check that <code>await</code> is present on both lines in <code>loadData</code>
Every district the same colour	The values cannot be read as numbers: a thousands separator, a stray space, or a placeholder such as <code>NA</code> . Check the attribute table
Map draws nothing and the console says Cannot read properties of null	A popup is reading an attribute that is <code>null</code> or named differently. Use the show helper from Part 7
Bounds are not valid	<code>fitBounds</code> was called on an empty selection. Check the <code>features.length > 0</code> guard in <code>draw</code>
Districts drawn in the ocean	Longitude and latitude are swapped in the source file
Hairline gaps between districts	Simplified without preserving topology. Re-run Simplify with that option enabled
Districts in the north are grey	Expected: Azad Kashmir and Gilgit-Baltistan were not covered by the 2023 census, so they have no population value
Can drag the map to another country	The <code>maxBounds</code> options from Part 5 are missing from <code>L.map</code>
Popup shows undefined	The property name in the popup does not match the attribute table
Page works locally, not when published	A path with a capital letter or a backslash. GitHub Pages is case-sensitive and needs forward slashes

Appendix B: Reference

Commands

<code>cd C:\webgis-lab</code>	go to the project folder
<code>code .</code>	open the folder in VS Code
<code>git status</code>	what has changed
<code>git add -A</code>	stage all changes
<code>git commit -m "..."</code>	save a snapshot
<code>git push</code>	upload to GitHub

```
git log --oneline
```

```
see the commit history
```

Developer tools

Tab	Use it for
Elements	Seeing the page as the browser built it, and editing CSS live
Console	Reading errors, and printing values with <code>console.log</code>
Network	Request sizes, status codes, timings, and throttling

Numbers worth remembering

Quantity	Value
Latitude range, Pakistan	about 24 to 37
Longitude range, Pakistan	about 61 to 77
Fifth decimal place	about 1.1 m
Tile size	256 by 256 pixels
EPSG:4326	WGS 84 degrees, for storage
EPSG:3857	Web Mercator metres, for display
EPSG:32642 and 32643	UTM zones 42N and 43N, for measuring
Area inflation at 33.6°	about 1.4 times

Optional, if you finish everything

1. Add a second basemap and a layer control to switch between them.
2. Highlight a district on mouseover by changing its outline weight.
3. Add a text box that filters districts by name as the user types.
4. Sort the drop-down by district count rather than alphabetically.
5. Replace the manual classification breaks with quantiles computed from the data.